

ENGINEERING EDUCATION · AI 101

AI for developers.

From auto-complete to autonomous agents — a working guide for engineers shipping software in the age of foundation models.

SECTION 01

Why now.

The moment AI stopped being a research story and started showing up in pull requests.

THE SHIFT

Software has a new **raw** **material.**

For fifty years, the basic unit of programming was a typed character. For the next decade, it is an instruction in plain English, interpreted by a model that already knows the answer's general shape. That changes the work, not the worker.

76%

BY THE NUMBERS

**Of professional developers
now use, or plan to use, AI
tools in their workflow.**

Up from 44% two years prior. The fastest tool-adoption
curve the discipline has ever seen.

WHERE THE LEVERAGE LANDS

Three things change about your **day**.

AI does not turn engineering into a single different job. It moves time from a few specific tasks to others — and the engineers who notice ship faster than the ones who don't.

01 · SPEED

Boilerplate compresses to seconds.

Scaffolds, glue, regex, fixtures, throwaway scripts. Things that used to be 20-minute jobs are now 20-second jobs.

02 · SURFACE AREA

You range further per day.

You touch unfamiliar languages, unfamiliar frameworks, and unfamiliar codebases with much less ramp-up cost.

03 · JUDGEMENT

Reading is the new writing.

Reviewing, naming, deciding what to keep and what to throw away — that work doesn't compress, and now dominates the day.

A TYPICAL TUESDAY

Yesterday's loop vs. the **new** loop.

WITHOUT AN AI PARTNER

Search, read, paste, adapt.

- Stack Overflow tabs open like a browser graveyard.
- Half a day to ramp on an unfamiliar SDK.
- Tests written last, written briefly, or written never.
- Refactors deferred because they are tedious, not because they are hard.

WITH AN AI PARTNER

Specify, review, iterate.

- Draft from a sentence; spend the saved time on edges.
- New SDK becomes a 20-minute conversation, not a half-day.
- Tests generated alongside the change, then audited.
- Tedious refactors become routine because the labour cost dropped.



The senior engineers on the team didn't get faster at typing. They got faster at deciding. The model writes a draft, they read it like a junior's PR, and the conversation moves to whether the design is right — not whether the syntax is right.



Engineering lead, automotive client
Internal interview • 2025

A WORKING DAY

Ten places AI already lives in your workflow.

If you've coded this week, you've probably used at least four of these without thinking of them as "AI." That is the point.

01

Inline code completion

The grey ghost-text you tab to accept.

03

Commit-message drafting

The diff goes in, a clean message comes out.

05

Cross-language translation

"Port this Python service skeleton to Go."

07

Codebase search and Q&A

Ask the repo, not a teammate, for the easy questions.

09

Documentation drafts

READMEs, ADRs, change-logs from the diff and the spec.

02

Chat assistant in the editor

"Why is this failing?" with the file as context.

04

Test scaffolding

Generated tables of cases for a function under test.

06

Ticket-to-PR drafts

Issue text in, a starter branch out.

08

SQL and query generation

Natural-language questions translated against your schema.

10

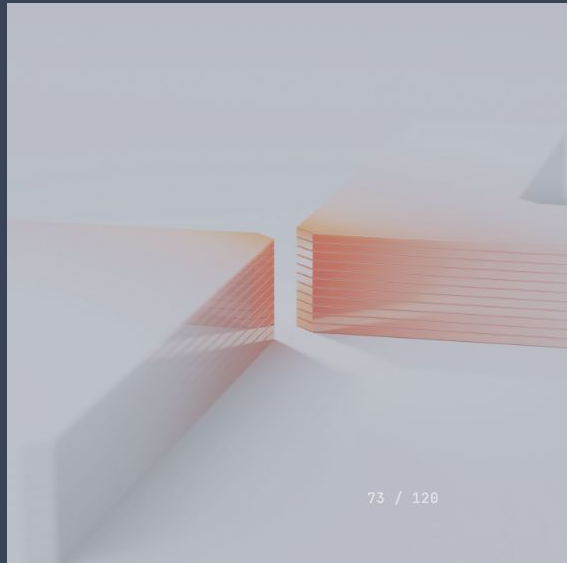
Code review pre-pass

Style, simple bugs, missed edge cases caught before the human reviewer.

SECTION 11

Intro to artificial intelligence.

A working tour of the field — from "what is AI, actually" to the families of models, the maths, and the seventy-year arc that produced this moment.



A WORKING DEFINITION

AI is the umbrella.
The interesting
stuff is underneath.

"Artificial intelligence" is a sixty-eight-year-old marketing term for any system that does something we used to think only humans could do. It is too broad to be useful in an engineering meeting.

Underneath sit three nested layers. **Machine learning** — algorithms that improve from data instead of being hand-coded. **Deep learning** — machine learning with neural networks that have many layers. **Foundation models** — very large neural networks trained on internet-scale data, then adapted to specific tasks.

When the press says "AI" today, they almost always mean the innermost ring. The outer rings have been shipping in production for decades — your spam filter, your recommendation feed, your fraud-detection rules. They still matter. They are still AI.

A SHORT, OPINIONATED HISTORY

Seven moments that got us **here.**

AI is not a new idea. The current moment is the latest beat in a much older story — useful context when someone tells you it's a bubble, or that it isn't.

1956

Dartmouth

The term "AI" is coined at a summer workshop. Optimism runs hot for two decades.

1986

Backprop

Rumelhart, Hinton, Williams put backpropagation on the map. Neural nets become trainable.

2012

ImageNet

AlexNet wins by a wide margin. Deep learning + GPUs becomes the default recipe for vision.

2017

Transformer

"Attention is all you need." The architecture under every modern LLM is published.

2020

GPT-3

Scaling laws stop being a hypothesis. 175B parameters; the API era of AI begins.

2022

ChatGPT

RLHF + a chat UI. A hundred million users in two months. The product moment.

2024+

Agents

Tool use, long context, multi-step reasoning. Models stop being text in / text out.

HOW MODELS LEARN

Four ways to **train** a model.

Almost every system you'll see — recommender, fraud check, LLM, robot policy — is one of these recipes, or a hybrid that mixes them.

01 · SUPERVISED

Labelled examples in, predictions out.

You give the model thousands of (input, correct answer) pairs. It learns the mapping. The workhorse of classical ML — spam filters, OCR, demand forecasting.

02 · UNSUPERVISED

No labels. Find structure.

Clustering, dimensionality reduction, anomaly detection. Tell me which customers are similar; tell me which transactions look weird.

03 · SELF-SUPERVISED

The data labels itself.

Hide part of the input; ask the model to predict it. This is how every modern LLM is pre-trained — predict the next token from the previous ones.

04 · REINFORCEMENT

Trial, error, reward.

An agent acts in an environment and receives a score. Used to fine-tune LLMs (RLHF) and to train game-players, robots, and recommendation policies.

A USEFUL DISTINCTION

Narrow AI vs. general AI.

NARROW AI · WHAT WE HAVE

One job, done well.

- Trained for a specific task — translation, classification, code completion.
- Outside that task, performance falls off a cliff. The model doesn't "know it doesn't know."
- Every production AI system in the world today is narrow AI, even when it feels broad.
- LLMs are unusually wide — but still narrow in the technical sense.

GENERAL AI · WHAT GETS DEBATED

Any cognitive task a human can do.

- One system that can learn arbitrary new tasks at human level, with no domain-specific training.
- Sometimes called AGI. Definitions vary; benchmarks are contested; timelines are guesses.
- Worth understanding the term so you can engage with the discussion — and disengage when it gets unproductive.
- For an engineering deck like this one: not on the roadmap.

INSIDE THE BOX

A neural network, demystified in five lines.

A neural network is a stack of matrix multiplications, with a non-linear function squashed between each layer. That's it. The "intelligence" is in the numbers inside those matrices — billions of them, set during training.

Forward pass: input vector $\times$ weights $\rightarrow$ activation $\rightarrow$ next layer $\times$ weights $\rightarrow$ activation $\rightarrow$... $\rightarrow$ output. **Loss:** how wrong was the output?

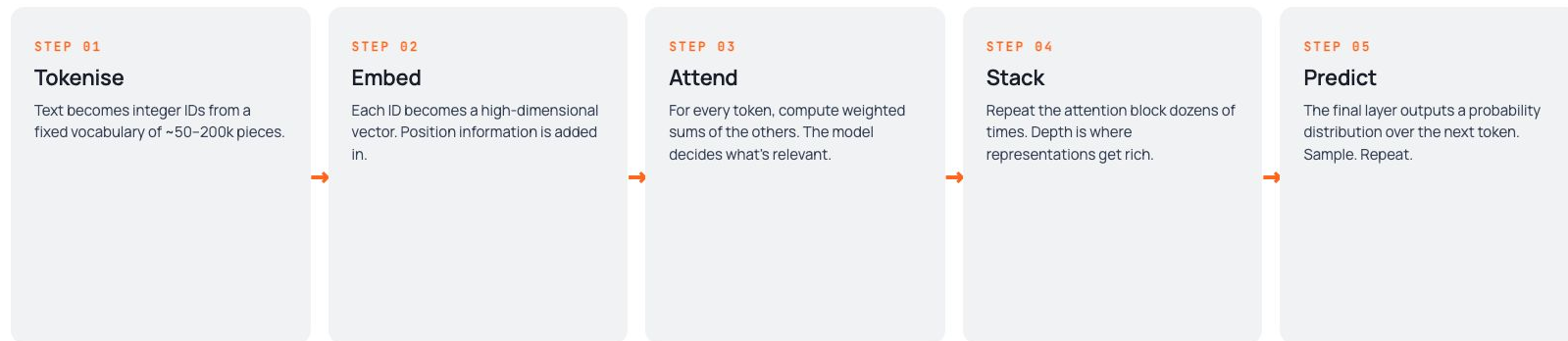
Backward pass: nudge every weight a little, in the direction that would have made the answer slightly less wrong. **Repeat** billions of times on billions of examples.

What's surprising is not the algorithm — it's that this brute-force recipe, scaled up enough, produces something that can write tests, translate languages, and explain a stack trace. Scale, in this field, has been the single biggest lever.

THE ARCHITECTURE UNDER EVERY MODERN LLM

What a **transformer** actually does.

If you remember nothing else: a transformer is a stack of "attention" layers that lets every token in the input look at every other token, weighted by relevance. That mechanism is the engine.



A WORKING MAP

Eight **capabilities** AI is already shipping.

Not predictions — features in production today. If you can name them, you can spot the right one for a problem in front of you.

01 · VISION

See and label.

Object detection, OCR, defect inspection, medical imaging. The most mature branch of deep learning.

02 · SPEECH

Listen and speak.

Real-time transcription, voice cloning, multilingual TTS. Latency and naturalness keep falling.

03 · LANGUAGE

Read, write, summarise.

The LLM family. The capability with the broadest product surface area in 2025.

04 · REASONING

Multi-step thinking.

Chain-of-thought and reasoning models. Slower, more expensive, dramatically better at hard tasks.

05 · GENERATION

Make new artefacts.

Images, video, audio, 3D, code. The medium-blurring branch of AI that drives most demos.

06 · RECOMMENDATION

Pick from a catalogue.

Feeds, search ranking, personalisation. The quiet giant — most of “AI” you experience daily.

07 · FORECASTING

Predict a number.

Demand, churn, fraud, time-to-failure. Classical ML continues to dominate here.

08 · CONTROL

Act in the world.

Robotics, autonomous systems, agentic software. The frontier where physics and policy meet.

A USEFUL CONTRAST

Classical ML or deep learning?

CLASSICAL ML

Hand-crafted features, simple models.

- Logistic regression, gradient boosting, random forests, SVMs. The toolkit that runs your business today.
- Trains in minutes on a laptop. Explains itself with feature importances.
- Wins on tabular data, low-data regimes, and anywhere regulators care about explainability.
- Boring in slide decks. Excellent in production.

DEEP LEARNING

Learn the features too.

- Neural networks that learn representations directly from raw data — pixels, audio, tokens.
- Trains for hours-to-weeks on specialised hardware. Hard to explain individual predictions.
- Wins on perception, language, and any domain where features are hard to hand-design.
- The right tool when classical methods plateau and you have a lot of unstructured data.

THREE NUMBERS, IN SCALE

The *scale* of a frontier model.

10¹²⁺

Parameters

Frontier dense models cross the trillion-parameter mark. Mixture-of-experts pushes effective scale further.

15^T

Training tokens

The most capable open models are trained on 10-20 trillion tokens – roughly the readable internet, several times over.

10²⁵

FLOPs

Total floating-point operations during training. The number that policy frameworks and chip-export rules now key off.



GlobalLogic
+ A Hitachi Group Company

In production · Healthcare client

A WORKING AI FEATURE

Vision + language, doing one job well.

For a continuous-glucose-monitor manufacturer, we shipped an in-app assistant that reads a glucose chart, answers questions in plain language, and surfaces an alert if a pattern looks risky. Two narrow AI capabilities — image understanding and language — composed for one calm clinical interaction.

→ The interesting AI is rarely a single model. It is composition.



The most important question to ask of any AI feature is not "what model" — it is "what data, what loop, and what does the user see when it's wrong?"



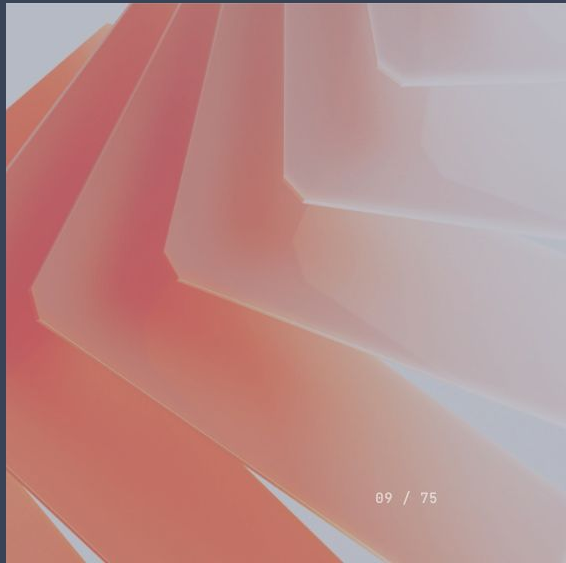
Andrey Khlobystov

VP of AI & Data, GlobalLogic

SECTION 02

How LLMs actually work.

A working mental model for engineers – enough to debug, not a research seminar.



FIRST PRINCIPLE

An LLM is a very good **autocomplete.**

It reads a sequence of tokens and predicts the most probable next token. Then it does that again. And again. That is the entire job description.

Everything else — chat, reasoning, tool use, code generation — is engineering on top of that single primitive. Knowing this changes how you debug it: when an answer goes wrong, ask what context the model could see, not what it “thought.”

It does not have memory between calls. It does not have access to your filesystem. It cannot run the code it writes. Anything that looks like those things is wired up around the model, not done by the model.

FROM YOUR PROMPT TO ITS ANSWER

The **five steps** of producing one token.

Repeat this pipeline a few thousand times and you have an answer.

STEP 01

Tokenize

Your text is split into sub-word units. "engineering" might become two or three tokens, not one.



STEP 02

Embed

Each token becomes a vector -- a list of numbers that places it in a high-dimensional space.



STEP 03

Attend

The transformer mixes information across the sequence so each position knows about the others.



STEP 04

Score

For every possible next token, the model assigns a probability based on everything seen so far.



STEP 05

Sample

One token is picked from that distribution. It is appended, and the loop runs again.

WORKING SCALE

The *three numbers* that describe a model.

~10_{B-1T}

Parameters

The trained weights. Bigger is generally smarter, but slower and more expensive per call.

~10_T

Training tokens

The corpus the model read during pretraining. The breadth of what it has "seen."

200_{k+}

Context window

How many tokens it can read at one time. The working memory of a single conversation.

TWO FLAVOURS OF THE SAME MODEL

Base models predict. **Instruct** models obey.

BASE / PRETRAINED

Pure text predictor.

- Trained to continue any text — code, prose, conversations, whatever the corpus contained.
- Asked "What is 2 + 2?", may continue with "is a question commonly asked of children..."
- Useful for fine-tuning, not for end users.

INSTRUCTION-TUNED / CHAT

Trained to follow.

- Further trained on instruction-response pairs and human preference data.
- Asked "What is 2 + 2?", answers "4" — because that is what helpful assistants do.
- This is what you actually call from your app — GPT-4o, Claude Sonnet, Llama 3 Instruct.

HOW A MODEL BECOMES USEFUL

Three stages from raw text to a **shipping** assistant.

You don't run any of these. But knowing the shape helps you reason about what a model will and won't do well.

01 · PRETRAINING

Reading the internet.

Trillions of tokens, weeks on thousands of GPUs. The model learns language, reasoning, and a great deal of code.

02 · SUPERVISED FINE-TUNING

Learning to be helpful.

Curated instruction-response pairs teach the model what a good answer to a request looks like.

03 · PREFERENCE LEARNING

Aligned to humans.

RLHF and similar methods nudge the model toward answers people prefer — clearer, safer, less verbose.



Working memory - The context window

A PRACTICAL FACT

Anything outside the context, the model cannot see.

The system prompt, the conversation so far, the file you pasted, the retrieved snippets — all of it has to fit in the window. When a model “forgets” earlier in a conversation, it has not forgotten. It can no longer see what was there.

→ Designing for context is now an engineering discipline.

SECTION 03

The toolkit.

A field guide to the categories that matter — and which one to reach for when.

THE FOUR SHAPES

Every AI dev tool is one of **four things**.

The marketing varies. The shape underneath is consistent.

01 · INLINE COMPLETER

Ghost text in the editor.

Predicts the next few lines as you type.
The lowest-friction surface; the one that disappears into your habits fastest.

02 · CHAT ASSISTANT

A conversation pane.

You ask, it answers, with awareness of the open file or repo. Useful for debugging, explaining, and small-to-medium edits.

03 · CODING AGENT

Autonomy with tools.

Reads files, runs commands, writes diffs, opens PRs. You give it a goal and it iterates — within bounds you set.

04 · API / SDK

Models in your code.

The raw inference call. What you reach for when you are building, not consuming, an AI feature.

A QUICK ORIENTATION

Tools you'll **actually** see this quarter.

Names move; categories don't. Use this as a map, not a leaderboard — and re-check before you commit a budget line.

TOOL	CATEGORY	BEST AT	WATCH OUT FOR
GitHub Copilot	Inline + chat	Day-to-day completion at scale	Plain features inside one editor
Cursor	IDE + chat + agent	Repo-aware edits across files	Lock-in to a forked editor
Claude Code	CLI agent	Multi-step terminal workflows	Power demands taste in prompts
JetBrains AI	IDE-native	Strong inside the JetBrains stack	Less ecosystem outside JetBrains
Codeium / Windsurf	Inline + agent	Free tier, on-prem options	Quality varies by language
OpenAI / Anthropic API	Raw inference	Building features into your product	You own the eval and the cost

THE MODEL LAYER

Foundation models you'll encounter.

A working roster — frontier and open. Pick by capability, latency, and where the data is allowed to live.

FRONTIER **GPT-4o, GPT-4.1 — OpenAI**

Strong all-rounder, broad ecosystem, excellent function calling.

FRONTIER **Gemini 1.5 / 2.x — Google**

Very large context windows; tight integration with Google data products.

OPEN **Llama 3 / 4 — Meta**

Strong open-weights baseline; the spine of most self-hosted stacks.

OPEN **DeepSeek, Qwen**

Strong on code and math; weights publicly available.

FRONTIER **Claude Sonnet / Opus — Anthropic**

Long context, careful reasoning, strong on code and nuance.

FRONTIER **Grok-2 — xAI**

Real-time data access, growing API surface, latecomer in tooling.

OPEN **Mistral / Mixtral**

Compact, fast, good for cost-constrained inference.

SPECIALIST **Code-specific models**

StarCoder, CodeLlama, DeepSeek-Coder — sharp on code, lighter on prose.



GlobalLogic
+ A Hitachi Group Company

In the editor · The new normal

WHAT GOOD LOOKS LIKE

The model is in the loop, not in the way.

Suggestions appear as you type. A side panel keeps a running conversation about the file. Diffs are proposed, not applied. The engineer is still the author of record — every accepted line crossed a human review.

→ [Pick a tool that respects the review step.](#)

A TYPICAL SESSION

Conversation, not command line.

You describe what you want, in the same words you'd use with a teammate. The model produces a proposal. You read, push back, accept the parts that hold up.

The model brings the syntax. You bring the system you're building.

~ chat session · cursor

// You

Add a debounce to the search input — 250ms,
keep the latest call only, cancel the in-flight one.

// Assistant

Here is the change to SearchBox.tsx:

```
const controllerRef = useRef<AbortController>()
const debounced = useDebounceCallback(async (q) => {
  controllerRef.current?.abort()
  const c = new AbortController()
  controllerRef.current = c
  await search(q, { signal: c.signal })
}, 250)
```

// You

Good. Add a test that fires three calls in 50ms and
asserts only one search() ran.

TWO SURFACES, TWO JOBS

Editor extensions or agent CLIs?

Both have a place. Neither replaces the other. The choice is about the shape of the task — not the tool.

Editor extensions excel when the work is tightly scoped: one file, a few functions, a refactor with obvious boundaries. The feedback loop is tight, the human stays in command, and risk is bounded by what you accept.

Agent CLIs earn their keep when the work crosses many files, runs commands, or includes a verification loop — generate, test, fix, repeat. You give a goal, the agent makes its own way there.

Rule of thumb: minutes to read the diff, use the editor; hours, hand it to an agent.

\$25B

CODING-TOOL MARKET, 2025

In annual spend on AI-assisted developer tooling — and growing roughly 40% year over year.

Most of that growth is the same engineers using the same tools more, not new buyers entering the market.

SECTION 04

Prompting like an engineer.

Not a magic spell. A specification — written for an extremely literal reader.

REFRAME

A prompt is a spec.

Treat it the way you treat a function signature. Inputs, outputs, constraints, examples. The vague request gets the vague answer; the spec-shaped request gets a spec-shaped answer. Most "AI is unreliable" complaints are missing-spec complaints.

WORKING PRINCIPLES

Eight rules that get you 80% of the way.

Internalised, these stop being prompt rules and become how you write any technical request.

01

State the role and the goal.

"You are a senior backend engineer. Your goal is to add a rate limiter to this Express app."

03

Show the shape of the answer.

"Return a single TypeScript file. No prose. Include JSDoc on the exported function."

05

Bring the context to the model.

Paste the file, the schema, the error. Don't expect it to guess from a description.

07

Ask for a plan before the change.

For non-trivial work, "what's the plan?" first, then "execute step 1." It catches mis-reads cheaply.

02

Name the constraints.

Language, framework, version, runtime, style guide. Anything you'd put in a coding standards doc.

04

Give one good example.

One worked example beats three sentences of description for almost any structured task.

06

Say what you don't want.

"Don't add new dependencies. Don't change unrelated files. Don't add comments explaining basic syntax."

08

Iterate, don't restart.

If the first answer is 80% there, point at the 20%. Re-prompting from scratch usually costs you the good 80%.

WORKED EXAMPLE

The difference between vague and specific.

Same intent, two prompts, different outcomes. Read them out loud and the gap is obvious.

A good prompt looks more like a Jira ticket than a Slack message.

~ prompt comparison

```
// ✗ Vague
"write a function to validate emails"

// ✓ Specific
"Write a TypeScript function `isValidEmail(s: string): boolean`.

Accept:
- RFC 5322-style addresses with quoted local parts
- Internationalised domain names

Reject:
- empty strings, whitespace-only strings
- addresses longer than 254 chars

Constraints:
- no external dependencies
- no regex longer than 80 chars

Return only the function and one short JSDoc block.
Provide three test cases as a separate code block."
```

ANATOMY OF A PROMPT

Three slots that do **most of the work.**

Once you've structured a prompt this way a few times, it becomes how you think about the request before you write a word.

01 · ROLE

Who is answering?

"A senior Go engineer." "A security reviewer." "A documentation editor." The role pins tone, depth, and defaults.

02 · TASK

What is being produced?

One sentence, one verb. "Refactor this function." "Add tests for the edge cases." "Translate this prompt into a curl command."

03 · CONSTRAINTS

What's the box around it?

Languages, libraries, length, format, things to avoid. The clearer the box, the less the model has to guess.

WHY SPECIFICITY PAYS

Same task. Two **worlds** of result.

VAGUE PROMPT

You get a stranger's defaults.

- Dependencies the project doesn't use.
- Style that doesn't match the codebase.
- Tests in the wrong framework, or no tests at all.
- Edge cases the model invented, not the ones you care about.

SPECIFIC PROMPT

You get something close to a PR.

- Code that fits your stack on the first try.
- Tests for the cases you specified.
- No new dependencies you didn't ask for.
- A diff small enough to review, large enough to merge.



Writing a good prompt is the same skill as writing a good ticket. The engineers who already write clear tickets had a head start the day they opened a chat assistant.

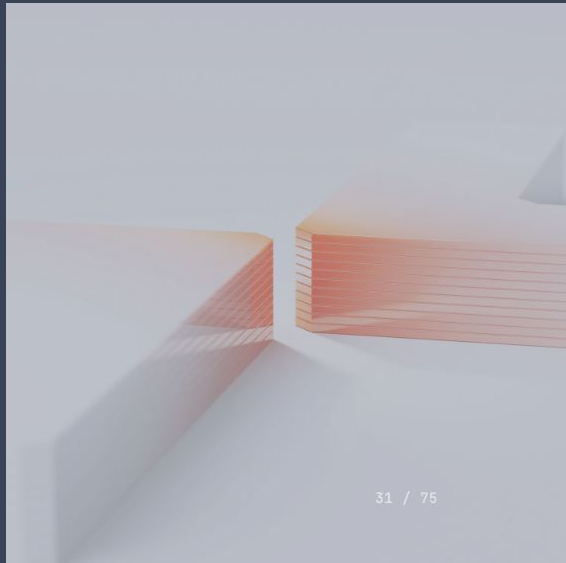


Staff engineer, fintech client
Internal interview · 2025

SECTION 05

Generating code that **ships**.

From "look what it wrote" to "look what we merged" — the patterns that survive code review.





GlobalLogic
+ A Hitachi Group Company

A new pair - The model and you

WORKING MODE

Treat the model as a tireless, slightly overconfident, junior engineer.

It will draft anything you ask. It will be plausibly wrong sometimes. It does not know your runtime constraints, your dependency graph, or which decisions are immovable. You do. The pairing works because you bring those.

→ The senior in the pair is still you.

REUSABLE PATTERNS

Eight code-gen patterns that **survive** review.

Each one is a phrase to keep on a sticky note next to your editor.

01

"Generate the test, then the code."

The test fixes the contract. The model fills in the implementation against a target it can execute against.

03

"Show me three options."

For design-y choices, ask for alternatives, then pick. Cheaper than guessing which one the model will commit to.

05

"Write the smallest change that passes."

Cap the model's ambition. Stops it from refactoring the whole file because it noticed something else.

07

"List the assumptions you just made."

An end-of-answer audit catches half the mistakes before they become bugs.

02

"Translate this, don't redesign it."

For ports across languages, lock the structure and ask only for translation. Suppress creative rewrites.

04

"Explain the diff, line by line."

Before you accept a change, force a narrated walkthrough. Most subtle bugs surface here.

06

"Write a worked example, then generalise."

Concrete-then-abstract is reliable. Abstract-first often produces something that compiles but doesn't fit.

08

"Re-read the requirements and try again."

The cheapest debugging move there is. Often produces a noticeably better second draft.

Generate the **tests** before the code.

Tests are an executable specification. The model is much better at writing code that passes a concrete test than at "writing the right code in general."

Bonus: the tests are a free review of your own intent.

~ vitest · slugify.test.ts

```
import { test, expect } from 'vitest'
import { slugify } from './slugify'

test('lowercases and joins with hyphens', () => {
  expect(slugify('Hello World')).toBe('hello-world')
})

test('collapses whitespace runs', () => {
  expect(slugify(' a b c ')).toBe('a-b-c')
})

test('strips punctuation, keeps unicode letters', () => {
  expect(slugify("It's working - café!")).toBe('its-working-café')
})

test('returns empty string for empty input', () => {
  expect(slugify('')).toBe('')
})
```

Refactors are now cheap.

The cost of "tedious but obvious" cleanups has dropped to near zero. The constraint is not effort. It is your willingness to read the diff carefully.

Always run the tests. Always.

~ prompt · refactor

```
// You
Refactor this function so that:
- it has no nested ternaries
- early returns replace the else branches
- the comparator is extracted as a named helper
- public behaviour and tests are unchanged
```

Show me the diff only. No prose.

```
// Assistant - diff
- export function sortJobs(a, b) {
-   return a.priority > b.priority ? -1 :
-     a.priority < b.priority ? 1 :
-     a.createdAt - b.createdAt
- }

+ function byPriorityThenAge(a, b) {
+   if (a.priority === b.priority)
+     return b.priority - a.priority
+   return a.createdAt - b.createdAt
+ }

+ export const sortJobs = byPriorityThenAge
```

TWO RELATIONSHIPS WITH THE MODEL

Accept-as-is vs. review-and-iterate.

ACCEPT-AS-IS

Speed today, debt tomorrow.

- You merge whatever the model produced because the tests went green.
- Style drifts. Patterns multiply. Ownership thins out.
- You discover the bug the model planted six weeks later, in production.
- It feels fast right up until it doesn't.

REVIEW-AND-ITERATE

The discipline that compounds.

- Every diff is read like a colleague's PR — by you, before you accept it.
- Style and patterns stay coherent because you are still the editor.
- You catch the bug at the keyboard, not in the on-call rotation.
- Slower per change in week one. Much faster, and safer, by month three.

26%

FIELD MEASUREMENT

**Faster task completion
among engineers using AI
assistants – concentrated
in less-experienced
engineers and routine work.**

Source: large-scale field study across thousands of developers, 2024. Senior engineers see smaller speed gains but bigger gains in scope per day.

SECTION 06

Testing, debugging, review.

The model as a quality partner — sometimes excellent, sometimes confidently wrong. Both useful, if you know which.

QA IN PRACTICE

Three numbers worth *tracking*.

+18_{pp}

Test coverage

Average uplift on legacy projects when AI is paired with a coverage gate over six weeks.

2.4_x

Bugs caught at review

When AI runs a first-pass review before the human reviewer, more issues are found earlier.

~30%

Time saved on flake triage

Models are good at scanning logs and reproducing flakes. They are not good at fixing the root cause alone.

WHERE THE MODEL EARNS ITS KEEP

Eight QA tasks AI does **well**.

Roughly in order of how much value most teams get out of them in their first month.

01

Edge-case enumeration

"Given this signature, what cases am I likely missing?" Almost always finds three you forgot.

03

Stack-trace explanation

Paste the trace; get a readable hypothesis. Treat it as a hypothesis, not a verdict.

05

Fixture and seed data

Realistic-looking test data on demand, in the schema you specify.

07

Review-comment first pass

Style, naming, missed null checks — picked up before a human reviewer's time is spent.

02

Property-based test seeds

Generators and invariants written from a function description.

04

Repro minimisation

"Reduce this 200-line failing test to the smallest case that still fails."

06

Snapshot diff explanation

Why this snapshot test broke, in human terms, with a prediction of which change caused it.

08

Migration safety scans

"Here is a schema change. What old code paths still assume the old shape?"

Ask for the cases you **didn't** think of.

Give the model the function and the existing tests. Ask "what's missing?" The answer is almost always more thoughtful than starting from scratch.

~ jest · parsePrice.test.ts

```
// You
Here is parsePrice() and three existing tests.
List edge cases that should be covered but aren't, then
write each missing test. Don't change the existing tests.
```

```
// Assistant
// Missing cases:
// 1. Locale: '1.234,56' (European thousands)
// 2. Currency prefix: '$1,234.56'
// 3. Negative as parenthetical: '(12.34)'
// 4. Scientific notation: '1.2e3'
// 5. Trailing whitespace + non-breaking space
```

```
test('parses European thousands separator', () => {
  expect(parsePrice('1.234,56', 'de-DE'))
    .toBeCloseTo(1234.56)
})
```

```
test('rejects scientific notation', () => {
  expect(() => parsePrice('1.2e3')).toThrow()
})
```

CODE REVIEW EVOLVES

Human review vs. **AI-assisted** review.

PURE HUMAN REVIEW

Slow, careful, expensive.

- The reviewer absorbs naming, style, and obvious bugs alongside the design.
- Bandwidth bound — only so many PRs per day per reviewer.
- Style nits and obvious bugs eat the time that should be spent on architecture.
- Quality scales linearly with reviewer attention.

AI-ASSISTED REVIEW

Two passes, two skills.

- The model does the first pass — naming, style, cheap bugs, missed cases.
- The human reviewer arrives with the easy stuff already flagged.
- Their time goes to design, intent, and the long-term shape of the codebase.
- Throughput rises without quality dropping — provided the human still signs off.



The unlock wasn't the model writing tests. It was the model arguing with us about which tests we should be writing — and being right often enough that we listened.



QA lead, media client

Internal interview • 2025

SECTION 07

From completion to **agency**.

Models that don't just answer — they read, run, retry, and report back. The shape of the work changes when the assistant can act.

WORKING DEFINITION

An **agent** is a model
with a loop and
tools.

A chat assistant answers once. An agent answers, observes the result, decides what to do next, and tries again — until it has met a goal or hit a budget.

The model is the reasoner. The loop is what makes it useful for multi-step work. The tools are what let it leave the chat box: read files, run shell commands, query a database, hit an API, open a PR.

Most "agentic" systems are this same pattern with different tool sets and guardrails wrapped around them.

THE INNER LOOP

Four steps, repeated until **done**.

Every agent framework you'll meet — LangGraph, OpenAI Assistants, Anthropic's tool use, your own — is some version of this loop.

STEP 01

Observe

Read the goal, the conversation so far, and the latest tool result. Build a picture of where things stand.



STEP 02

Plan

Decide what to do next. Often this is "call a tool"; sometimes it is "ask the user"; sometimes it is "I am done."



STEP 03

Act

Run the chosen tool with chosen arguments. The result comes back as text the model can read on the next turn.



STEP 04

Reflect

Did that get us closer? If yes, continue. If no, change the plan. If the budget is spent, hand back to the human.

WHAT AN AGENT DOES FOR YOU

Four kinds of work to **delegate**.

If a task fits one of these shapes, an agent is probably the right tool. If not, a chat assistant likely is.

01 · INVESTIGATE

Read the codebase, answer the question.

"Where do we handle webhook retries?" — agent reads files, traces calls, replies with citations.

02 · IMPLEMENT

Plan, edit, test, iterate.

Given a ticket, the agent drafts the change, runs the tests, fixes what breaks, opens a PR.

03 · MIGRATE

The same change, many places.

"Replace this deprecated call across 40 files." Bulk, mechanical, well-suited to a tireless executor.

04 · OPERATE

Triage, alert, hand off.

Watch a queue, summarise incidents, draft responses. Keep the human in the loop for anything destructive.

UNDER THE HOOD

What "tool use" actually looks like.

The model emits a structured request. Your code runs the function. The result is fed back into the conversation. Repeat.

It is much more boring than the demos suggest. That is the good news.

~ anthropic / openai · tool use loop

```
// 1. You declare tools the model may call
const tools = [
  { name: 'read_file',
    description: 'Read a file by path',
    input_schema: { path: 'string' } },
  { name: 'run_tests',
    description: 'Run vitest, return output',
    input_schema: {} },
]

// 2. The model replies with a tool_use block
{ type: 'tool_use',
  name: 'read_file',
  input: { path: 'src/auth/middleware.ts' } }

// 3. You execute it, return the result
const result = await readFile('src/auth/middleware.ts')

// 4. Loop until the model says it's done
while (response.stop_reason !== 'tool_use') { /* ... */ }
```



In production · An agent at work

A REAL EXAMPLE

"Migrate this service to the new auth library."

The agent reads the service, plans the change, edits across files, runs the tests, fixes the failures, runs the tests again, opens a PR with a summary. Forty minutes of unattended work that used to be a half-day.

→ Bound it: timeout, file allowlist, no merging.

WHERE EACH ONE WINS

Copilot or **agent**?

COPILOT — YOU DRIVE

Tight loop, small bets.

- You hold the keyboard. The model proposes; you accept line by line.
- Best for tasks measured in minutes — one function, one file, one obvious change.
- Risk is bounded by what you visibly accept.
- The dominant mode for most of your day, even after agents land.

AGENT — MODEL DRIVES

Loose leash, bigger jobs.

- You set a goal and bounds; the agent chooses the steps.
- Best for tasks measured in hours — multi-file changes, mechanical migrations, repeated triage.
- Risk has to be designed in — sandboxes, allowlists, dry-runs, no auto-merge.
- The right tool when the loop is bigger than your attention span.

SECTION 08

Putting AI into your product.

From "I called an API" to "we shipped a feature." The patterns, the call, the trade-offs that decide if it works.

WHERE TO POINT A MODEL

Three jobs LLMs are genuinely good at.

If your feature idea fits one of these, you're in the strong-suit zone. If it doesn't, look harder before reaching for a model.

01 · TRANSFORM

Reshape text and structure.

Summarise, translate, extract, classify. The bread-and-butter — and where most successful AI features live.

02 · GENERATE

Produce a draft from a brief.

Email replies, code stubs, descriptions, options. The model gives you a starting point; the user keeps the steering wheel.

03 · REASON OVER CONTEXT

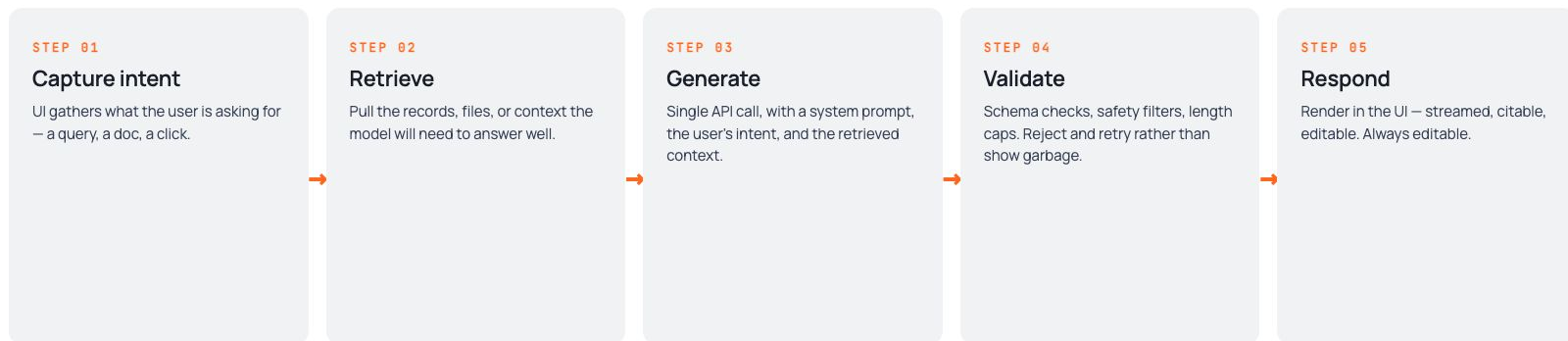
Answer with documents.

Q&A grounded in your data. The model is the lawyer; the documents are the case file. Retrieval makes this usable.

A TYPICAL PIPELINE

Most AI features look like the **same** pipeline.

If you can sketch each step, you can build the feature. If you can't, you don't yet have the spec.



A PATTERN CATALOGUE

Eight features **nine** in ten apps end up needing.

Naming the pattern is half the work — half of “what should we build with AI?” is rediscovering one of these.

01 · SUMMARISATION

Long in, short out.

Threads, tickets, transcripts, documents.
Almost always the first feature shipped.

02 · CLASSIFICATION

Label this.

Tag, route, prioritise. Replaces brittle
regex pipelines, often at lower cost.

03 · EXTRACTION

Pull out fields.

Free-form text in, structured JSON out.
The most “honest day’s work” pattern
there is.

04 · DRAFTING

Empty page, gone.

Replies, descriptions, summaries — first
drafts that the user finishes.

05 · Q&A ON DOCS

RAG with citations.

Manuals, runbooks, contracts. The
serious version of “search.”

06 · TRANSLATION

Across languages or formats.

Natural language, code, schema, dialect. A
reliable workhorse.

07 · COACHING

Inline guidance.

Suggestions on what the user is writing or
doing, while they do it.

08 · SEARCH REWRITE

Better queries on the way in.

Expand, disambiguate, translate user
phrases into proper search queries.

THE SMALLEST USEFUL CALL

An LLM feature is, at its core, **twenty lines**.

Everything else is around that call: retrieval, validation, error handling, observability. The call itself stays small.

~ node · summarise.ts

```
import Anthropic from '@anthropic-ai/sdk'
const client = new Anthropic()

export async function summarise(ticket: Ticket) {
  const res = await client.messages.create({
    model: 'claude-sonnet-4-5',
    max_tokens: 300,
    system: `You summarise support tickets for engineers.
Return three bullets: problem, what was tried, ask.`,
    messages: [{
      role: 'user',
      content: renderTicket(ticket),
    }],
  })

  const text = res.content[0].text
  return parseBullets(text)
}
```

AN HONEST DECISION

Use the API, or buy a **platform**?

DIRECT API

Maximum control.

- You own the prompts, the data flow, the eval, and the bill.
- Switching models is a config change, not a renegotiation.
- Right answer when the feature is core, sensitive, or strategic.
- Wrong answer if the team has no appetite for ops on day two.

MANAGED PLATFORM

Velocity and a paved road.

- Retrieval, evals, observability, safety filters in one box.
- Faster to first user value; usually pricier per call.
- Right answer for non-core features and small teams.
- Wrong answer when the platform's roadmap diverges from yours.



Case study · Financial services

A WORKING FEATURE

An assistant that drafts the response, never the verdict.

For a top-twenty bank, we built an in-app assistant for service agents. It reads the case file, drafts a reply in the bank's voice, and never sends without an agent's edit. Average handle time fell 22%; quality scores rose.

→ [The model accelerates. The human decides.](#)

DESIGN BUDGETS

Every AI feature has *three budgets*.

< 2_s

Latency

Time-to-first-token shapes whether the feature feels assistive or sluggish. Stream responses; pre-fetch context.

~ \$/call

Cost

Track cost per session, not per call. The cheapest call is the one you cached or didn't need to make.

> 90%

Quality

Set a ground-truth threshold before launch. Below it, do not ship. Use evals, not vibes, to decide.

SECTION 09

Giving the model **your** data.

RAG is not a magic word. It is the engineering pattern that makes a generic model useful inside your product.



WORKING DEFINITION

RAG is "open-book"
for the model.

Retrieval-Augmented Generation. The model takes its exam open-book — you find the relevant pages, hand them in with the question, and ask for an answer that cites them.

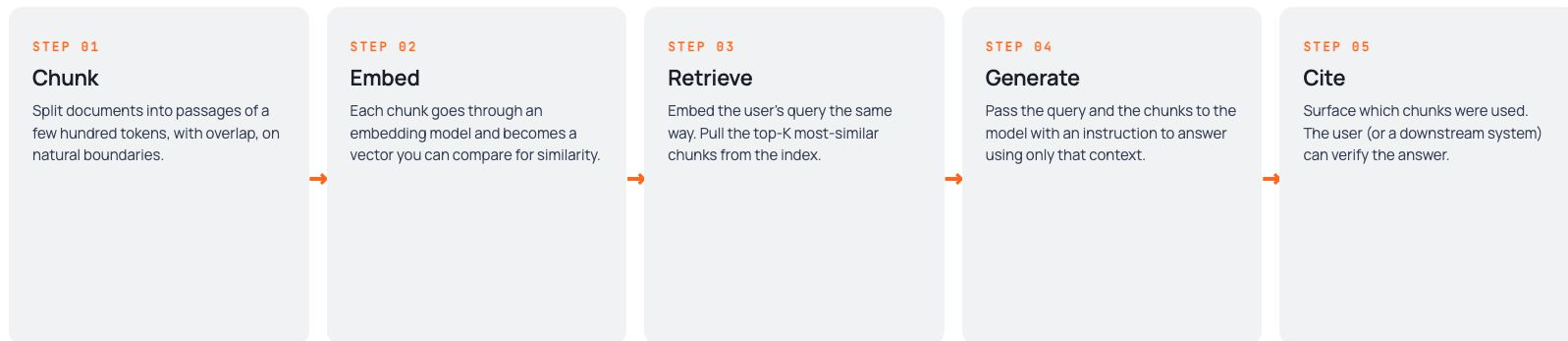
It works because models are far better at reading from a context than at recalling facts. Your runbook, your codebase, your contract — turn them into retrievable chunks, fetch the right ones, and the same model becomes accurate in your domain.

It also works because retrieval is debuggable. When the answer is wrong, you can look at exactly which chunks went in. That is more than you can say about most ML systems.

THE FIVE-STEP PIPELINE

From a folder of documents to a **working** answer.

Steps one and two run offline. Steps three through five run on every query.



WHERE VECTORS LIVE

Four shapes of **vector** store.

Pick on operational fit, not on benchmark scores. The fastest store is the one your team can already run.

01 · POSTGRES EXTENSION

pgvector

Vectors next to your relational data. Best when the team already runs Postgres and the corpus is moderate.

02 · PURPOSE-BUILT

Pinecone, Weaviate, Qdrant

Hosted services tuned for vector workloads. Pay-as-you-go; minimal ops.

03 · SEARCH ENGINES

Elastic, OpenSearch

Hybrid keyword + vector retrieval. Strong when text search and semantic search both matter.

04 · EMBEDDED

SQLite-vss, Lance

Vectors inside the application. Right answer for small corpora, edge devices, single-tenant tools.

The **shortest** RAG you can ship.

Forty lines of code, a vector store, an embedding model, and a chat model. Everything past this is reranking, hybrid search, and tasteful caching.

~ node · ask.ts

```
async function ask(question: string) {  
  // 1. embed the question  
  const qVec = await embed(question)  
  
  // 2. fetch top-K similar chunks  
  const chunks = await db.query(`  
    SELECT id, source, text  
    FROM docs  
    ORDER BY embedding <=> $1  
    LIMIT 6  
  `, [qVec])  
  
  // 3. ask the model with that context  
  const res = await model.messages.create({  
    system: `Answer using only the provided sources.  
           Cite them as [1], [2], etc.  
           If the answer isn't there, say so.`,  
    messages: [{  
      role: 'user',  
      content: render({ question, chunks }),  
    }],  
  })  
  
  return { answer: res.content[0].text, chunks }  
}
```

A FREQUENT QUESTION

Should we **fine-tune** or use RAG?

FINE-TUNING

Teach a new behaviour.

- Use when you need a specific style, format, or skill the base model lacks.
- Costs are upfront and recurring as the base model improves.
- Hard to keep current — every data change is a new training run.
- Right answer for tone, structure, narrow classification tasks.

RAG

Hand it the right facts.

- Use when the model needs your data to answer, but already knows how to write.
- Cost is per-query inference; updating "knowledge" means re-indexing.
- Easier to debug — you can see what was retrieved.
- Right answer for the overwhelming majority of internal-knowledge use cases.

THINGS THAT GO WRONG

Eight failure modes RAG **brings** with it.

Most "the AI hallucinated" bugs are actually one of these wearing a clever costume.

01

Bad chunking

Splits cut sentences in half. Retrieval finds fragments; answers stay shallow.

03

Top-K blindness

You retrieved six chunks. The right answer was at rank seven. Always test with adversarial queries.

05

Permission leakage

The retriever pulls a chunk the user shouldn't see. Filter by ACL before, not after, the model sees it.

07

Pure semantic search

Hybrid search (keyword + vector) almost always beats vectors alone. Don't skip the BM25.

02

Wrong embedding model

The model that wrote the corpus doesn't have to be the one that reads it. Match domains.

04

Stale index

Documents change; the index doesn't. The model confidently cites yesterday's policy.

06

Conflicting sources

Two chunks contradict. The model picks one. Surface the conflict, don't paper over it.

08

No "I don't know"

If the prompt doesn't permit refusal, the model invents. Always allow "the sources don't say."

SECTION 10

Shipping responsibly.

Evals, safety, and the operational disciplines that turn an AI demo into a feature you'd put your name on.

FIRST COMMITMENT

Evals are tests.

If your code has unit tests, your AI feature has evals. The same instinct: a small, curated set of inputs with expected behaviour, run on every change. Without them, you cannot ship a model swap, a prompt change, or a context tweak with any confidence at all.

WHAT "PRODUCTION" ACTUALLY MEANS

The same three numbers,
watched in production.

P95

Latency budget

Streaming first token, total time to complete, retry overhead. Set them; alert when they're missed.

\$/req

Cost budget

Cost per request, per user, per feature. Watch the long tail – runaway agents and runaway prompts hide there.

Win_{rate}

Quality budget

Eval pass rate over time. Model swaps, prompt edits, retrieval changes – all gated on this number.

PRE-LAUNCH CHECKLIST

Ten safety questions before **flipping** the feature on.

Not exhaustive, not compliance advice. A working floor a senior engineer would expect to hear answers to.

01

What does the model see?

System prompt, user input, retrieved context. Catalogue every input — and every secret it might brush against.

02

What can the model do?

If it has tools, list them. The blast radius of the worst plausible call is your real risk surface.

03

Where does data leave?

Region, vendor, retention. Match against contracts and regulation before the first query, not after.

04

How are PII fields handled?

Redacted, tokenised, or sent in the clear? Each option has consequences. Make the choice deliberately.

05

What about prompt injection?

Treat retrieved content and user input as untrusted. Never let it override your system instructions.

06

What's the refusal path?

If the model can't or shouldn't answer, what does the user see? Design that surface deliberately.

07

What are the rate limits?

Per user and per feature. Both abuse and runaway loops show up here first.

08

How do we audit a bad output?

Logs of inputs, outputs, retrieved context, model version. With privacy controls. Don't ship without them.

A WORKING EVAL

The **simplest** eval that's worth running.

Curate twenty inputs you care about. Capture the ideal behaviour. Score every change against them. That's it. The fancier evals are just elaborations of this loop.

~ node - evals/summarise.eval.ts

```
const cases = readFixtures('tickets/golden/')

for (const c of cases) {
  const out = await summarise(c.input)

  // rule-based checks - cheap, deterministic
  expect(out.bullets).toHaveLength(3)
  expect(out.bullets.join(' ')).not.toMatch(/SSN|password/i)

  // LLM-as-judge - graded against a rubric
  const grade = await judge({
    rubric: 'Does this summary correctly state the user's ask?',
    expected: c.expected,
    actual: out.bullets,
  })

  expect(grade.score).toBeGreaterThanOrEqual(4)
}
```

THE ARC OF AN AI FEATURE

Day one vs. day ninety.

DAY ONE

It's a demo.

- One prompt. One model. One context shape. Looks magical to leadership.
- Edge cases live in the demo script, not the test set.
- Cost and latency are not yet a budget — they're a guess.
- Roll-out feels easy because nothing about it has been stressed.

DAY NINETY

It's a system.

- Versioned prompts, evals on every change, observability you trust.
- A kill switch and a fallback path that have both been used in anger.
- A model-swap drill that takes a config change, not a quarter.
- The team can talk credibly about quality, cost, and risk in the same meeting.



The AI feature didn't fail because the model was wrong. It failed because we shipped it without the muscle for evals, and a year later nobody could safely change anything.



Director of engineering, healthcare client
Internal interview • 2025

SECTION 12

Prompt engineering, deeper.

Past the basics: techniques, sampling parameters, structured outputs, and the prompt patterns that survive contact with production.

WHERE PROMPTING LIVES

Three **modes** of prompting you'll write this year.

The tactics differ by mode. A 5-line chat prompt is not the same artefact as a 500-line system prompt running in production.

01 · CONVERSATIONAL

Throwaway, in-chat.

You're talking to Claude or ChatGPT. Iterate quickly, accept the result, move on. Write like you'd brief a smart contractor.

02 · TEMPLATED

Shipped inside a feature.

A prompt with placeholders that the app fills in. Versioned, reviewed, eval-tested. Treat it like code, because it is.

03 · AGENTIC

System prompts that run loops.

Long, structured instructions for an autonomous agent. Tool definitions, escalation rules, refusal behaviour, budget bounds.

ZERO-SHOT, ONE-SHOT, FEW-SHOT

How **many examples** should you show?

A simple progression. Try the cheapest one first; only climb the ladder when quality demands it.

STEP 01

Zero-shot

Just describe the task. No examples. Works surprisingly often with capable models — try it first.



STEP 02

One-shot

One worked example. Pins down format and tone in a way no description can match.



STEP 03

Few-shot

Three to ten examples covering the variation you actually see. The reliable tier for production templates.



STEP 04

Dynamic-shot

Retrieve the most relevant 3 examples from a library at runtime. The bridge between prompt and RAG.

CHAIN-OF-THOUGHT

Ask the model to **show its work**.

For multi-step problems, instructing the model to reason step-by-step before answering reliably improves accuracy. Modern reasoning models do this internally; smaller models still benefit from the explicit nudge.

~ system prompt - cot.md

Role

You are a senior backend engineer reviewing a bug report.

Method

Before you answer, work through the problem in this order:

1. Restate the symptom in your own words.
2. List the three most plausible causes, ranked by likelihood.
3. For the top cause, describe the exact evidence in the stack trace that supports it.
4. Propose the smallest fix that addresses the cause without introducing new behaviour.

Output format

Reasoning: <your steps 1-4 above>

Answer: <the proposed fix, in 3 sentences>

Now do it

{{stack_trace}}

REACT PROMPTING

Reason, then act, then read the result.

The pattern that turned LLMs into agents. Each step is "Thought → Action → Observation," looping until the model emits "Final Answer." Most agent frameworks are this pattern formalised.

~ trace · react-loop.txt

```
# Question
Which version of the auth service is in prod, and is it
above the patched range for CVE-2025-12345?

# Iteration 1
Thought:    I need the deployed version. Then I need to
            check the CVE's affected range.
Action:     kubectl_get(deploy=auth, field=image)
Observation: ghcr.io/acme/auth:1.4.6

# Iteration 2
Thought:    I have 1.4.6. Now check the CVE bulletin.
Action:     cve_lookup(id="CVE-2025-12345")
Observation: Affected: < 1.4.5. Patched: 1.4.5+.

# Final answer
auth-1.4.6 is above the patched range; not vulnerable.
```

A PATTERN LIBRARY

Eight **moves** that show up in good prompts.

Naming the move is half the work. Once you can spot it in someone else's prompt, you start using it in your own.

01 · PERSONA

"You are a ..."

Sets register and vocabulary. Strongest effect on tone; weaker on accuracy.

02 · CONSTRAINTS

Boundaries first.

"Use only the provided sources." "Do not invent endpoints." Make the floor explicit.

03 · SCHEMA

Lock the output shape.

Specify JSON keys, field types, and refusal cases. Then validate.

04 · SELF-CRITIQUE

"Before you reply, check..."

Internal QA pass: factuality, schema, tone. A surprising amount of polish for one extra line.

05 · DECOMPOSITION

Split the task.

globallogic.com
First do A. Then B. Only then write the answer." Helps weaker models keep their footing.

06 · FEW-SHOT

Examples beat description.

One canonical example per branch. Care about. Cover the boring case

07 · NEGATIVE EXAMPLE

"Do not produce X."

Show what failure looks like. Often more than positive instruction alone.

08 · REFUSAL PATH

"If unsure, say so."

Give the model permission to decline. The 90 / 120 cheapest hallucination preventer there is.

STRUCTURED OUTPUTS

Make the output a contract.

Free-form text is fine for chat. For features, ask for JSON that matches a schema, validate it, and reject-and-retry on failure. Most providers now support a native structured-output mode that does this for you.

~ ts · extract-issue.ts

```
const schema = {
  type: 'object',
  required: ['severity', 'component', 'summary'],
  properties: {
    severity: { enum: ['low', 'med', 'high'] },
    component: { type: 'string' },
    summary: { type: 'string', maxLength: 280 },
    tags: { type: 'array', items: { type: 'string' } },
  },
},

const res = await model.create({
  response_format: { type: 'json_schema', schema },
  system: 'Extract structured fields from the issue.',
  messages: [{ role: 'user', content: issue.body }],
})

// Parse + revalidate. Don't trust the wrapper.
const data = validate(schema, JSON.parse(res.text))
```

THREE KNOBS, SET DELIBERATELY

The *sampling* parameters you'll touch.

0.0–1.0

Temperature

How random is the next-token pick? 0 for extraction and code; 0.7 for drafting; 1.0 only for brainstorming.

0.9

Top-p

Nucleus sampling — restrict to tokens covering the top-p mass. Usually leave at 0.9; tune temperature instead.

512–4k

Max tokens

An upper bound on output length. Set tight for extraction; generous for drafts. Always set something.

THINGS TO STOP DOING

Ten prompt **anti-patterns**.

Spotted in real client code, named here so you don't have to discover them in production.

01

Politeness padding

"Please could you, if you don't mind, kindly..." Models don't reward courtesy. Be direct.

03

Implicit format

"Reply nicely." Define the format. Markdown? JSON? Three bullets? Be specific.

05

Forbidding without examples

"Don't be verbose" is weaker than "reply in < 80 words, like the example below."

07

The kitchen sink

Twenty paragraphs in the system prompt. Each adds noise. Trim ruthlessly; eval after.

09

No refusal path

The prompt forbids saying "I don't know." So the model invents. Always permit refusal.

02

Stacked superlatives

"You are the best, most expert, world-class..." The fifth adjective doesn't move accuracy. Pick one.

04

Mixing roles

System prompt with the user's question pasted in. Use the role separators the API gives you.

06

Truth-claim instructions

"Always be 100% accurate." It will write that and be wrong anyway. Add citations and refusal paths instead.

08

Magic phrases

"Take a deep breath." "Think harder." Cargo cult. Replace with structure.

10

Untracked changes

Prompt edits without version control. Then quality regresses and nobody knows why. Treat prompts like code.

A WORKING TEMPLATE

The **shape** of a production system prompt.

Six blocks, in this order. You'll deviate, but having a default skeleton makes prompts reviewable and diff-able.

~ prompts/triage.md

1. Role

You are a triage assistant for the SRE team.

2. Goal

Classify the incoming alert and propose a next action.

3. Inputs you'll receive

- alert.json (Datadog payload)
- context.md (last 3 incidents on this service)

4. Constraints

- Use only information in the inputs.
- If signal is insufficient, return action="page_human".
- Never propose a destructive action automatically.

5. Output

JSON matching schema in /schemas/triage.json.

Severity ∈ {low, med, high, sev1}.

6. Examples

<example pair 1>

<example pair 2>

<example with action="page_human">

A BUDGETING DECISION

Long context or retrieval?

LONG CONTEXT

Stuff it all in.

- Frontier models accept hundreds of thousands of tokens. The temptation is real.
- Costs grow linearly; latency grows worse than linearly. Quality degrades on irrelevant context.
- Useful when the right context is small, fixed, and you don't have retrieval infra.
- Beware "lost in the middle" — models attend less reliably to the centre of long inputs.

RETRIEVAL

Fetch only what's relevant.

- Pull the top-K relevant chunks at query time. Less context, more precision, lower cost.
- Auditable — you can show the user exactly what informed the answer.
- Adds operational surface — an index, an embedding model, a refresh job.
- Almost always the right answer once the corpus is bigger than a manual.



The job of a prompt is the same as the job of a spec — to make the next person who reads it agree with you about what success looks like. The next person, in this case, is the model.



Practice lead, applied AI
GlobalLogic — internal handbook

SECTION 13

AI across the lifecycle.

From the first user interview to the post-incident review — eight phases of software, each touched by AI in concrete, measurable ways.

A REFRAMING

AI doesn't
replace a phase.
It **compresses**
the loop.

The SDLC didn't disappear. Requirements still need eliciting, designs still need debate, code still needs review, releases still need a runway.

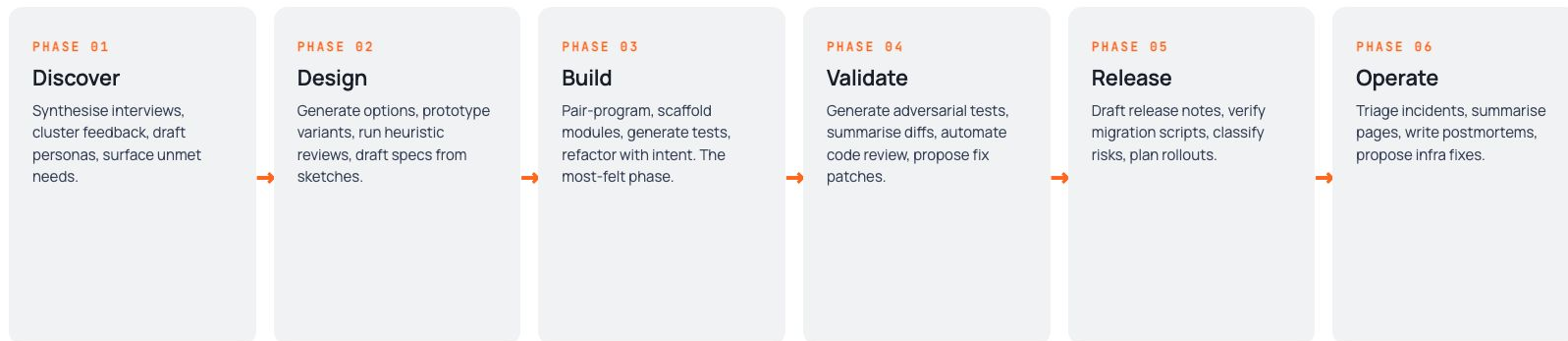
What's changed is the cycle time of each step. A user interview becomes a draft persona in twenty minutes. A whiteboard becomes a prototype before lunch. A failing test becomes a passing test in the same PR. A pager becomes a triage summary in seconds.

The teams that get the most out of AI in the SDLC are not the ones that automate end-to-end. They're the ones that find the slowest beat in their loop, point a model at it, and keep the human in charge of the verdict.

SIX PHASES, SIX LEVERAGE POINTS

Where AI **plugs in** across the lifecycle.

Each phase has its own bottleneck. The right AI feature is the one that unblocks that specific beat.



EARLY PHASES · IN DETAIL

Discover, design, build, **validate**.

Concrete features, with the kind of impact a senior PM would actually believe.

01 · DISCOVER

Interview synthesis.

Cluster 40 transcripts into themes with quotes. Two days of work in twenty minutes. Quality matches a careful PM; speed does not.

02 · DESIGN

Spec from sketch.

Hand the model a Figma frame, get a draft user story with acceptance criteria. Designer keeps editing rights.

03 · BUILD

Inline pair-programming.

Multi-line completions, refactors, and conversational fixes. The most-adopted AI feature in the industry today.

04 · VALIDATE

Adversarial tests.

Generate edge-case inputs from the spec — boundary, malformed, hostile. Catches the cases reviewers always miss.

LATE PHASES · IN DETAIL

Release, operate, secure, **support.**

The phases where AI is least talked-about, and often where it has the highest leverage.

05 · RELEASE

Release notes from the diff.

Auto-summarise merged PRs into customer-facing notes. Reviewer keeps the pen.

06 · OPERATE

Pager triage.

Cluster alerts, propose root cause, draft a runbook step. Human is on the keyboard for anything destructive.

07 · SECURE

Threat-model assistant.

Summarise the design, suggest STRIDE risks, propose tests. Security keeps the verdict.

08 · SUPPORT

Tier-1 deflection.

Answer with citations from your docs and ticket history. Hand off when confidence drops.

Specify the contract, let the model write the tests.

The shape of one of the most reliable wins in the SDLC.

The human writes the spec. The model expands it into the test cases — including the edge cases the human would forget.

~ tests/billing.spec.ts · ai-augmented

```
// Spec written by the engineer
describe('computeInvoiceTotal', () => {
  // 1. zero items returns 0
  // 2. mixed currencies throw
  // 3. negative quantities throw
  // 4. discount cannot exceed subtotal
  // 5. tax applied after discount

  // → 14 tests generated by the assistant ↓
  it('returns 0 for empty items', () => { ... })
  it('throws on mixed currencies', () => { ... })
  it('throws on negative qty', () => { ... })
  it('caps discount at subtotal', () => { ... })
  it('applies tax to discounted subtotal', () => { ... })
  it('rounds to two decimals on JPY (no decimals)', () => { ... })
  it('preserves precision on chained discounts', () => { ... })
  // ...7 more, including locales the human had forgotten
})
```

A SIDE-BY-SIDE

Traditional SDLC vs. AI-augmented.

TRADITIONAL

Sequential, phase-bounded.

- Each phase has a hand-off, a document, a meeting, and a queue.
- Throughput limited by the slowest specialist on the loop.
- Quality known late — most defects found in QA or production.
- Predictable, auditable, slow.

AI-AUGMENTED

Parallel, conversational.

- Discovery, design and a working prototype overlap in days, not weeks.
- Feedback shifts left — adversarial tests in PR, threat-model in design, summary in PR.
- Specialists move from "doing" to "directing and reviewing." Throughput climbs.
- New disciplines required: evals, prompt review, AI guardrails — these are the new SDLC artefacts.



Case study · Industrial software

FROM DISCOVERY TO RELEASE, FASTER

One AI assistant per phase, one engineer in charge.

For a global manufacturing platform, we wired one AI assistant into each SDLC phase: requirements clustering, design critique, pair coding, test expansion, release-note drafting, and on-call triage. Lead time from idea to production fell 38%; defect-escape rate held flat.

→ [Compress the loop, don't shorten the verdict.](#)

THINGS THAT GO WRONG

Eight **pitfalls** we keep seeing.

Patterns from a hundred-plus client engagements. They do not feel cliché until you've stepped on each one.

01

"AI strategy" instead of AI bets

An AI-themed roadmap with no measurable bet on it. Pick one workflow per quarter; staff it; measure it.

03

No evals, no rollback

The feature ships; quality drifts; nobody can prove it. Evals on day one, or not on day one at all.

05

Skipping the human-in-the-loop

An auto-approving pipeline in week two. A scary outage in week six. Always a human for destructive actions.

07

One-shot enablement

A two-hour training, then silence. Instead: pairing weeks, internal channels, a visible champion per team.

02

Tool sprawl

Every team gets a different copilot. Standardise on a small menu, with sanctioned data flows, before adoption ossifies.

04

Productivity as the only metric

Lines of code accepted is not impact. Track lead time, defect rate, and developer experience.

06

Data exposure by accident

Source code, customer data, secrets — sent to a model whose retention policy nobody read. Get the policy first.

08

Forgetting the org chart

The hardest part is rarely the model. It's procurement, legal, security, change management. Plan for it.

MEASURE THE RIGHT THREE THINGS

The SDLC *scoreboard*.

-38%

Lead time

Idea to production. The single best aggregate signal that AI is actually compressing the loop.

flat

Defect-escape rate

Bugs found by users. Should hold or improve. If it rises, you are accepting more than you are reviewing.

+18_{pts}

Developer NPS

The qualitative half. AI tools are doing their job when engineers say the work is more interesting.

A GOVERNANCE MODEL THAT WORKS

Permissive at the edge, **strict** at the core.

Treat AI usage like data classifications: not one rule, but a small ladder, applied based on what the model touches.

Tier 1 – public: chat with public data, learning, prototypes. Permissive; encourage exploration. **Tier 2 – internal:** code reviews, drafting, summarising – sanctioned tools, retention controls, no customer PII.

Tier 3 – regulated or production: approved models only, full eval coverage, audit trails, change-controlled prompts, no autonomous destructive actions.

Get the ladder agreed once across security, legal, and engineering – then push the work back into teams. Governance is the floor, not the ceiling.



The teams winning with AI in the SDLC are not the ones with the boldest demos. They are the ones with the most boring discipline — versioned prompts, real evals, kill switches, and a leader who can spell DORA.



Engineering excellence lead
GlobalLogic — practice forum, 2025

SECTION 14

Agentic AI, deeper.

Past the demo: autonomy levels, multi-agent topologies, the protocols underneath, and the design principles that keep agents useful in production.

A USEFUL LADDER

Five **levels** of agent autonomy.

Lifted from automotive. Surprisingly clarifying for software. Pick a level deliberately; design the guardrails for that level only.

L0

Suggester

Proposes an action; human types it. Inline completion lives here.

L1

Assistant

Executes a single tool call when asked. Reads a file, runs a query, summarises an output.

L2

Operator

Plans and runs a small loop under supervision. Most "agent mode" IDEs sit here today.

L3

Delegate

Owens a multi-step job from a goal — migrations, triage runs, scheduled work. Human reviews outcomes.

L4

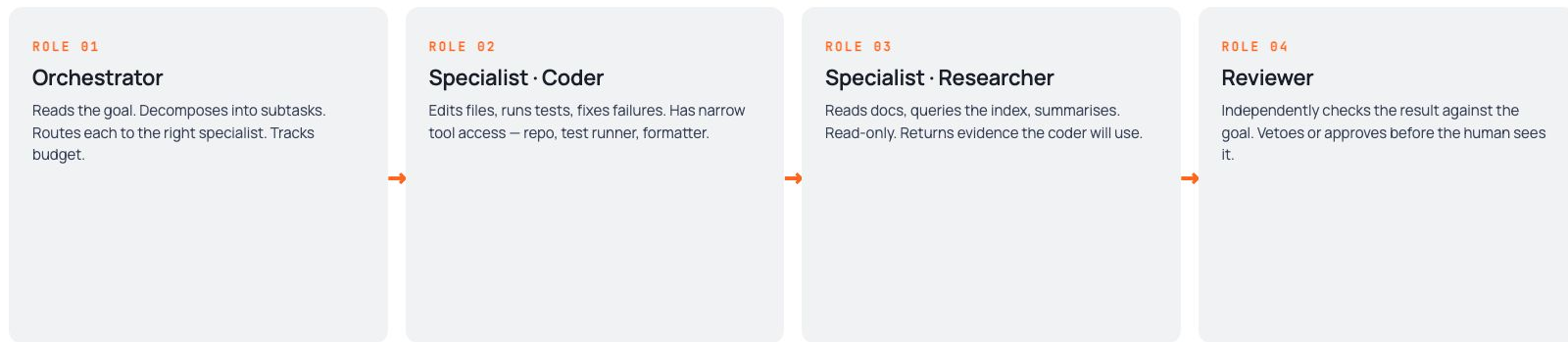
Autonomous

Operates without per-task approval. Reserved for narrow, well-bounded, reversible domains.

A MULTI-AGENT TOPOLOGY

One **orchestrator**, several **specialists**.

The most common production shape. A planning agent decomposes the goal; specialists do the work; a reviewer agent sanity-checks the output before it hits the human.



INSIDE A WORKING AGENT

Four **parts**, four interfaces.

If your agent doesn't have a clear answer for each of these, it isn't an agent — it's a long prompt with hopes attached.

01 · PLANNER

Decides what to do next.

The model itself, with a system prompt that defines the loop, the budget, and the stopping condition.

02 · TOOLS

Lets it act on the world.

Typed function definitions. Each is its own product surface — name, schema, description, error model, latency.

03 · MEMORY

Carries state across turns.

Short-term: the conversation. Long-term: a vector store or scratchpad. Episodic: per-task notes the agent re-reads.

04 · EVALUATOR

INDEPENDENT CHECK ON OUTPUT QUALITY. EITHER A SECOND MODEL, A RULE SET, OR BOTH. THE BRAKE PEDAL.

A WORKING AGENT LOOP

The whole pattern, in thirty lines.

The frameworks add ergonomics — but the core loop is small enough to write by hand, and you should, at least once. Doing so demystifies a lot of the hype.

~ agent.ts · minimal ReAct

```
const tools = { readFile, runTests, writePatch }
const history: Msg[] = [{ role: 'user', content: goal }]

for (let i = 0; i < MAX_STEPS; i++) {
  const res = await model.create({
    system: SYSTEM,
    messages: history,
    tools: describe(tools),
  })

  if (res.stop_reason === 'end_turn') {
    return res.text // agent says it's done
  }

  for (const call of res.toolCalls) {
    const out = await tools[call.name](call.args)
    history.push({ role: 'tool', name: call.name, content: out })
  }
}

throw new BudgetExceeded('agent did not converge')
```

THE PLUMBING

Four **protocols** agents talk over.

Standardising on these is what turns one-off agent demos into reusable building blocks across a team.

01 · FUNCTION CALLING

The native tool interface.

Every major model exposes typed function definitions. The lowest-level, most-portable mechanism.

02 · MCP

Model Context Protocol.

An open standard for plugging tools, resources and prompts into any agent. Reduces N*M connector explosion.

03 · A2A

Agent-to-agent.

Emerging conventions for agents to discover and call each other. Useful when one team's agent needs another team's.

04 · CODE EXECUTION

The universal tool.

Give the agent a sandboxed runtime; let it write the rest of its tools. Powerful – and the one with the largest blast radius.

A DESIGN CHOICE, NOT A FASHION

One agent or several?

SINGLE AGENT

One loop, full context.

- One model holds the goal, the history, the tool calls, the verdict.
- Easiest to debug. Cheapest to run. Best default.
- Fails when the task spans roles with conflicting prompts (writer + reviewer + executor).
- Reach for multi-agent only when a single prompt visibly cannot hold both jobs.

MULTI-AGENT

Specialised loops, mediated.

- Each agent has a focused prompt and a narrower tool surface.
- Lets you separate "the doer" from "the checker" — independent verdicts catch what one model would rationalise.
- Operationally heavier — more prompts, more evals, more failure modes between agents.
- Right answer for jobs with clearly distinct roles, like research → code → review.

A WORKING SET OF PRINCIPLES

Ten **rules** for agents that don't bite.

Each was learned the hard way on a real engagement. None of them are exotic — they're the discipline that keeps an agentic feature shippable.

01 Reversible by default

Every action the agent can take should have a backout. Destructive operations get an extra approval step.

03 Narrow the tool surface

The smaller the tool list, the steadier the agent. Prefer five sharp tools over fifty fuzzy ones.

05 Make failure cheap

Sandboxes, ephemeral branches, dry-run modes. The agent must be allowed to fail without anyone calling support.

07 Log everything

Every prompt, tool call, observation, and decision. With privacy controls. Without logs you cannot debug an agent.

09 Plan a refusal

What does "I can't do this safely" look like? Design that path; rehearse it; ship it.

02 Bound the loop

Maximum steps, wall-clock, and dollar budget. The model never decides when "enough is enough."

04 Type the inputs

Every tool has a JSON schema. Every output is validated. No "looks like a path" — just paths.

06 Evaluate the agent, not just the model

Eval the loop end-to-end on golden tasks. Model-only evals miss orchestration bugs.

08 Surface the thinking

Show the user what the agent is doing in real time. A black-box agent is a trust-killer.

10 Keep the human accountable

Even at L4 autonomy, somebody owns outcomes. Make that name visible in the system, the dashboard, and the org chart.



The hard problem of agents isn't intelligence — it's blast radius. Engineers who can reason carefully about what an agent is allowed to touch are the ones who get to ship agents at all.



Director of platform engineering
Public-sector client, 2025

CLOSING

The road ahead.

Where this is heading, what stays human, and where to point your week tomorrow.

A TAKE, NOT A FORECAST

The job stays the same. The *medium* changes.

Engineering will still be about the same things — clear thinking under uncertainty, taste in trade-offs, picking what's worth building. The unit of work is moving from typed characters to specified intent. Engineers who treat that as good news ship more, sooner.

Engineering impact, with AI.

CONTINUE LEARNING

learn.globallogic.com

INTERNAL CHANNEL

[#ai-for-engineers](#)

QUESTIONS

ai-education@globallogic.com